

CORS 问题核心原理及后端配置

一、CORS 问题核心原理

CORS（跨域资源共享）是浏览器的安全机制，当前端页面从**不同域名/端口/协议**请求后端资源时，浏览器会触发跨域校验：

- 简单请求（GET/POST，Content-Type 为 `application/x-www-form-urlencoded` / `multipart/form-data` / `text/plain`）：直接发送请求，后端需返回合法的 CORS 响应头，否则浏览器拦截响应。
- 预检请求（OPTIONS）：非简单请求（如 PUT/DELETE、自定义头、JSON 格式）会先发送 OPTIONS 预检，验证后端是否允许跨域，通过后才发真实请求。

CORS 报错本质是**后端未返回符合要求的跨域响应头**，或预检请求未被正确处理。

二、核心解决思路：后端配置跨域响应头

所有跨域问题的核心解决点在**后端**（前端仅能做辅助规避，无法根本解决），需返回以下关键响应头：

响应头	作用
Access-Control-Allow-Origin	允许的跨域源（如 <code>https://frontend.com</code> 或 <code>*</code> 通配所有源）
Access-Control-Allow-Methods	允许的请求方法（如 GET,POST,PUT,DELETE,OPTIONS）
Access-Control-Allow-Headers	允许的自定义请求头（如 Token,Content-Type）
Access-Control-Allow-Credentials	是否允许携带 Cookie（true/false，需与前端 <code>withCredentials</code> 匹配）
Access-Control-Max-Age	预检请求缓存时间（秒，避免频繁发 OPTIONS）

1. 不同后端框架的配置示例

(1) Node.js/Express

使用 `cors` 中间件（推荐）：

Code block

```
1  const express = require('express');
2  const cors = require('cors');
3  const app = express();
4
5  // 基础配置（允许所有源，仅测试用）
6  app.use(cors());
7
8  // 生产环境精准配置
9  app.use(cors({
10     origin: 'https://your-frontend.com', // 指定允许的前端域名
11     methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'], // 允许的方法
12     allowedHeaders: ['Content-Type', 'Token'], // 允许的自定义头
13     credentials: true, // 允许携带Cookie
14     maxAge: 86400 // 预检缓存1天
15 }));
16
17 // 手动处理（不推荐，易漏）
18 app.use((req, res, next) => {
19     res.setHeader('Access-Control-Allow-Origin', 'https://your-frontend.com');
20     res.setHeader('Access-Control-Allow-Methods', 'GET,POST,PUT,DELETE,OPTIONS');
21     res.setHeader('Access-Control-Allow-Headers', 'Content-Type,Token');
22     res.setHeader('Access-Control-Allow-Credentials', 'true');
23     // 处理预检请求
24     if (req.method === 'OPTIONS') {
25         return res.sendStatus(200);
26     }
27     next();
28 });
```

(2) Java/Spring Boot

方式1：注解（单个接口/类）

Code block

```
1  import org.springframework.web.bind.annotation.CrossOrigin;
2  import org.springframework.web.bind.annotation.GetMapping;
3  import org.springframework.web.bind.annotation.RestController;
4
5  @RestController
6  // 类级别跨域配置
```

```

7  @CrossOrigin(
8      origins = "https://your-frontend.com",
9      methods = {RequestMethod.GET, RequestMethod.POST, RequestMethod.PUT,
10     RequestMethod.DELETE},
11     allowedHeaders = {"Content-Type", "Token"},
12     allowCredentials = "true",
13     maxAge = 86400
14 )
15 public class DemoController {
16     @GetMapping("/api/data")
17     // 接口级别 (覆盖类配置)
18     @CrossOrigin(origins = "https://your-frontend.com")
19     public String getData() {
20         return "success";
21     }
22 }

```

方式2：全局配置（推荐）

Code block

```

1  import org.springframework.context.annotation.Bean;
2  import org.springframework.context.annotation.Configuration;
3  import org.springframework.web.cors.CorsConfiguration;
4  import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
5  import org.springframework.web.filter.CorsFilter;
6
7  @Configuration
8  public class CorsConfig {
9      @Bean
10     public CorsFilter corsFilter() {
11         CorsConfiguration config = new CorsConfiguration();
12         // 允许的源 (生产环境不要用*, 配合credentials需指定具体域名)
13         config.addAllowedOrigin("https://your-frontend.com");
14         // 允许携带Cookie
15         config.setAllowCredentials(true);
16         // 允许的方法
17         config.addAllowedMethod("*"); // 或精准指定 GET, POST 等
18         // 允许的头
19         config.addAllowedHeader("*"); // 或精准指定 Content-Type, Token 等
20         // 预检缓存时间
21         config.setMaxAge(86400L);
22
23         UrlBasedCorsConfigurationSource source = new
24         UrlBasedCorsConfigurationSource();
25         // 对所有接口生效
26         source.registerCorsConfiguration("/**", config);
27     }
28 }

```

```
26         return new CorsFilter(source);
27     }
28 }
```

(3) Python/FastAPI

Code block

```
1  from fastapi import FastAPI
2  from fastapi.middleware.cors import CORSMiddleware
3
4  app = FastAPI()
5
6  app.add_middleware(
7      CORSMiddleware,
8      allow_origins=["https://your-frontend.com"], # 允许的源
9      allow_credentials=True, # 允许Cookie
10     allow_methods=["GET", "POST", "PUT", "DELETE", "OPTIONS"], # 允许的方法
11     allow_headers=["Content-Type", "Token"], # 允许的头
12     max_age=86400, # 预检缓存时间
13 )
14
15 @app.get("/api/data")
16 async def get_data():
17     return {"msg": "success"}
```

(4) Nginx 反向代理（前端部署/网关层解决）

若无法修改后端代码，可通过 Nginx 转发请求，在 Nginx 配置中添加跨域头：

Code block

```
1  server {
2      listen 80;
3      server_name api.your-domain.com;
4
5      location / {
6          # 转发到后端真实地址
7          proxy_pass http://127.0.0.1:8080;
8
9          # 跨域配置
10         add_header Access-Control-Allow-Origin "https://your-frontend.com"
always;
11         add_header Access-Control-Allow-Methods "GET,POST,PUT,DELETE,OPTIONS"
always;
12         add_header Access-Control-Allow-Headers "Content-Type,Token" always;
13         add_header Access-Control-Allow-Credentials "true" always;
```

```
14         add_header Access-Control-Max-Age "86400" always;
15
16         # 处理预检请求
17         if ($request_method = OPTIONS) {
18             return 204;
19         }
20     }
21 }
```

三、前端辅助注意事项

1. 避免非法配置：

- 若后端设置 `allowCredentials: true`，`Access-Control-Allow-Origin` 不能用 `*`，需指定具体域名；
- 前端使用 `fetch` / `axios` 携带 Cookie 时，需开启 `withCredentials`：

Code block

```
1 // axios 示例
2 axios.get('https://api.your-domain.com/data', {
3     withCredentials: true
4 });
5
6 // fetch 示例
7 fetch('https://api.your-domain.com/data', {
8     credentials: 'include'
9 });
```

2. 规避跨域的临时方案（非解决，仅绕过）：

- 开发环境：前端脚手架（Vue/React）配置代理（webpack devServer）：

Code block

```
1 // vue.config.js
2 module.exports = {
3     devServer: {
4         proxy: {
5             '/api': {
6                 target: 'https://api.your-domain.com',
7                 changeOrigin: true,
8                 pathRewrite: {'^/api': ''}
9             }
10        }
11    }
```

```
12    };
```

- 生产环境不建议用此方案，仅开发时临时规避。

四、常见坑点

1. **预检请求（OPTIONS）被拦截**：后端需放行 OPTIONS 请求（返回 200/204），不能拦截或返回业务逻辑；
2. **多个域名允许**：后端需动态判断 Origin 是否在白名单，再返回对应 `Access-Control-Allow-Origin`（不要直接返回 `*`）；
3. **自定义头未配置**：前端传自定义头（如 Token）时，后端需在 `Access-Control-Allow-Headers` 中包含该头；
4. **HTTPS 跨域**：HTTPS 域名跨 HTTP 域名时，浏览器会严格拦截，需统一协议（均为 HTTPS）。

五、排查方法

1. 打开浏览器开发者工具 → Network → 找到跨域请求 → 查看 Response Headers 是否包含上述 CORS 头；
2. 若为预检请求，查看 OPTIONS 请求的响应状态（应为 200/204）和响应头；
3. 控制台报错关键词：`No 'Access-Control-Allow-Origin' header`（缺核心头）、`Credentials flag is 'true' but 'Access-Control-Allow-Origin' is '*'`（Credentials 与 `*` 冲突）。

总结

CORS 问题的根本解决是**后端返回合规的跨域响应头**，前端仅需配合调整请求配置（如 `withCredentials`）；开发环境可通过代理临时规避，生产环境必须通过后端/Nginx 配置跨域头，且遵循“最小权限原则”（不滥用 `*`，指定具体源/方法/头）。

（注：文档部分内容可能由 AI 生成）